Comparing models

Hypothesis testing

$$Y_i \sim Bernoulli(\pi_i) \quad \log\left(\frac{\pi_i}{1-\pi_i}\right) = \beta_0 + \beta_1 \text{GRE}_i$$

We want to know if there is actually a relationship between GRE score and grad school admission.

> How can I express this as null and alternative hypotheses about one or more model parameters?

$H_0: \beta_1 = 0$ (no relationship)

$H_A: \beta_1 \neq 0$ (some relationship)

Steps in hypothesis testing

- Specify hypotheses (e.g. $H_0: \beta_1 = 0$, $H_A: \beta_1 \neq 0$)
- Check assumptions (model diagnostics)
- Calculate a test statistic (t-statistic, z-statistic, F-statistic, ...)
- Calculate p-value
- Draw a conclusion

Test statistic

$H_0: \beta_1 = 0$

$H_A: \beta_1 \neq 0$

```
m1 <- glm(admit ~ gre, family = binomial,
          data = grad_app)
summary(m1)
```

$$Z = \frac{\hat{\beta}_1 - 0}{SE(\hat{\beta}_1)} = \frac{0.003582}{0.000986} = 3.633$$

```
...
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.901344   0.606038  -4.787 1.69e-06 ***
## gre          0.003582   0.000986   3.633  0.00028 ***
...
```

Which part of this output should I use as my test statistic for the hypothesis test?

p-value

```
m1 <- glm(admit ~ gre, family = binomial,
          data = grad_app)
summary(m1)
```

```
...
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.901344   0.606038  -4.787 1.69e-06 ***
## gre          0.003582   0.000986   3.633  0.00028 ***
...
```

p-value

> Which part of this output gives me the p-value for my hypothesis test?

p-value

```
m1 <- glm(admit ~ gre, family = binomial,
          data = grad_app)
summary(m1)
```

```
...
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.901344   0.606038  -4.787 1.69e-06 ***
## gre          0.003582   0.000986   3.633  0.00028 ***
...
```

> What distribution are we using to calculate that p-value?

standard normal $N(0, 1)$

p-value

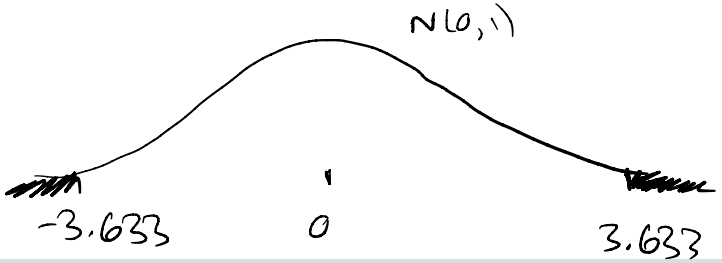
```
m1 <- glm(admit ~ gre, family = binomial,
          data = grad_app)
summary(m1)
```

```
...
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.901344   0.606038  -4.787 1.69e-06 ***
## gre          0.003582   0.000986   3.633  0.00028 ***
...
```

Our goal today: why is a N(0, 1) distribution appropriate for calculating the p-value here?

$H_0: \beta_1 = 0 \quad H_A: \beta_1 \neq 0$

Calculating p-values



> What *is* a p-value?

p-value = probability of "our data or more extreme" if H_0 is true

p-value = $P(Z > 3.633 \text{ or } Z < -3.633 \mid \beta_1 = 0)$

↑ probability; 3.633: observed test statistic calculated from $N(0,1)$ distribution

$N(0,1)$: distribution of Z under H_0

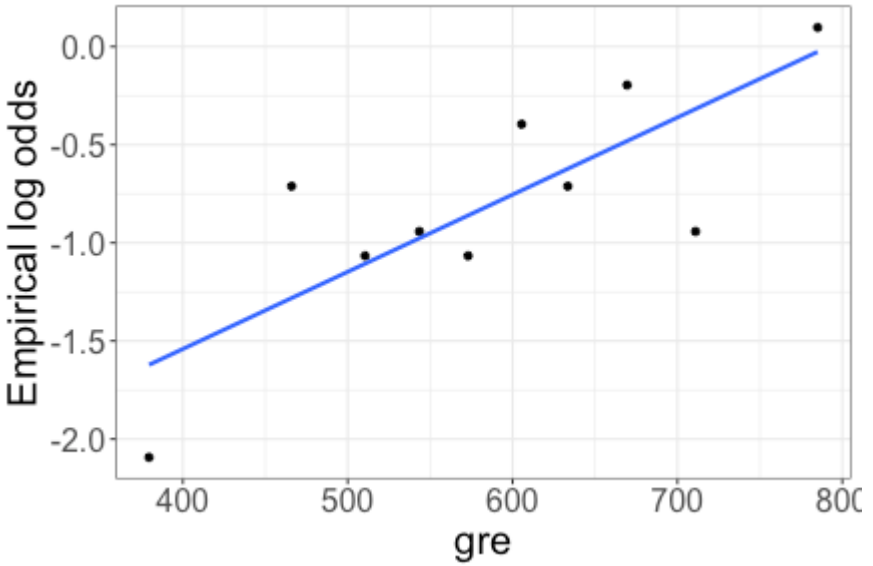
Null distribution

Null distribution: distribution of our test statistic *under* H_0

- We need to know how our test statistic behaves if the null hypothesis is true
- That is, how would the test statistic vary from sample to sample?

Observed relationship

```
logodds_plot(grad_app, admit ~ gre, num_bins = 10)
```



> What would this plot look like if H_0 were true?

Making H_0 true

- Currently, there may be a relationship between GRE and Admit
- To remove this relationship: randomly shuffle the GRE values

Currently

GRE	Admit
400	0
410	0
500	0
⋮	
700	1
720	1

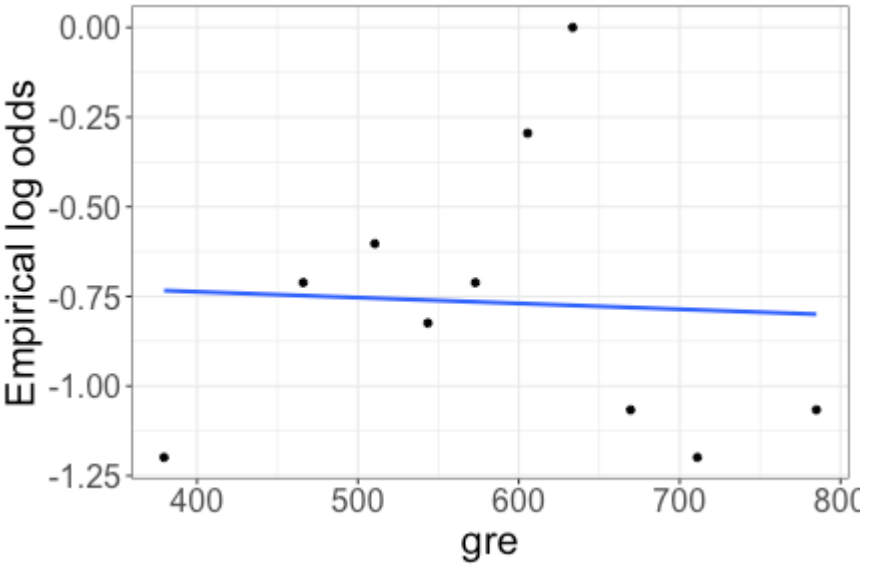
Shuffled

GRE	Admit
400	1
410	0
500	1
520	1
⋮	
700	0
720	1

Making H_0 true

```
grad_app_perm <- grad_app |>
  mutate(gre = sample(grad_app$gre, n(), replace = F))

logodds_plot(grad_app_perm, admit ~ gre, num_bins = 10)
```



Making H_0 true

```
grad_app_perm <- grad_app |>
  mutate(gre = sample(grad_app$gre, n(), replace = F))

perm_mod <- glm(admit ~ gre, family = binomial,
                  data = grad_app_perm)

summary(perm_mod)
```

$z = -0.128$

```
...
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.6950976  0.5568946  -1.248    0.212
## gre         -0.0001195  0.0009307  -0.128    0.898
...
```

> What is my test statistic for the permuted data (for which H_0 is *true*)?

Making H_0 true

```
grad_app_perm <- grad_app |>
  mutate(gre = sample(grad_app$gre, n(), replace = F))

perm_mod <- glm(admit ~ gre, family = binomial,
                  data = grad_app_perm)

summary(perm_mod)
```

```
...
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.6950976  0.5568946  -1.248    0.212
## gre         -0.0001195  0.0009307  -0.128    0.898
...
```

> Does this give me the *distribution* of my test statistic under H_0 ?

Repeating the process

```
nreps <- 1000 # number of repetitions
z_stats <- rep(NA, nreps) # place to store results

head(z_stats)
```

repeating value # repetitions

```
## [1] NA NA NA NA NA NA
```

Repeating the process

```
nreps <- 1000 # number of repetitions
z_stats <- rep(NA, nreps) # place to store results

for(i in 1:nreps){
  # do something
}
```

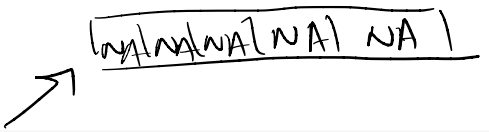
For loop: example

```
for(i in 1:5){
  print(i)
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

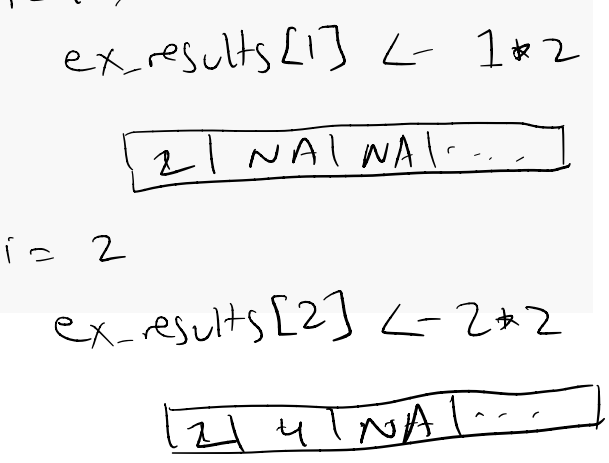
i = 1
    print(1)
i = 2
    print(2)
    ⋮
i = 5
    print(5)

For loop: example



```
ex_results <- rep(NA, 5)
for(i in 1:5){
  ex_results[i] <- i*2
}

ex_results
```



```
## [1]  2  4  6  8 10
```

```
ex_results[1]
```

```
## [1] 2
```

```
ex_results[2]
```

```
## [1] 4
```

Repeating the process

```
nreps <- 1000 # number of repetitions
z_stats <- rep(NA, nreps) # place to store results

for(i in 1:nreps){
  # do something
}
```

> What needs to happen inside our for loop?

- shuffle data
- fit model
- calculate Z-statistic
- store results

Repeating the process

```
nreps <- 1000 # number of repetitions
z_stats <- rep(NA, nreps) # place to store results

for(i in 1:nreps){
  grad_app_perm <- grad_app |>
    mutate(gre = sample(grad_app$gre, n(),
                        replace = F))

  perm_mod <- glm(admit ~ gre, family = binomial,
                  data = grad_app_perm)
  z_stats[i] <- summary(perm_mod)$coefficients[2,3]
}
```

shuffling & saving data

fit model w/ shuffled dat

store results for ith iteration

getting Z-statistic

Class activity

https://sta214-f25.github.io/class_activities/ca_12.html

- Work independently or with a neighbor
- Submit HTML on Canvas when done